

Design pattern

MỤC LỤC

Lời nói đầu	3
A. Tổng quan về Design pattern	4
I. Vấn đề trong thiết kế phần mềm hướng đối tượng.....	4
II. Lịch sử design pattern	4
III. Design pattern là gì?.....	5
B. Hệ thống các mẫu design pattern	6
I. Hệ thống các mẫu	6
1. Nhóm Creational	6
2. Nhóm Structural	6
3. Nhóm Behavioral.....	6
4. Sơ lược chuẩn của mẫu	6
5. Quy tắc biểu diễn mẫu trong UML.....	7
II. Nội dung các mẫu Design pattern	8
1. Abstract Factory	8
2. Builder	12
3. Factory Method	13
4. Prototype	15
5. Singleton.....	16
6. Adapter	18
7. Bridge	19
8. Composite.....	20
9. Decorator	23
10. Façade.....	24
11. Flyweight.....	26
12. Proxy	28
13. Chain of Responsibility	30

14. Command	33
15. Interpreter.....	35
16. Iterator.....	38
17. Mediator.....	40
18. Memento	43
19. Observer.....	45
20. State	46
21. Strategy	46
22. Template Method	47
23. Visitor	48
C. Ứng dụng design pattern trong thực tế phân tích thiết kế	
phần mềm hướng đối tượng	50
I. Framework và idiom.....	50
II. Kiến trúc Add – Ins	51
D. Các mẫu thiết kế hiện đại.....	52
I. Gamma Patterns.....	52
II. Entity Pattern (datasim).....	52
III. Concurrent Patterns.....	52
E. Xây dựng ứng dụng Chess sử dụng Design pattern.....	53
F. Tài liệu tham khảo	53
I. Sách.....	53
II. Địa chỉ website	53

Lời nói đầu

Design pattern là một kỹ thuật dành cho lập trình hướng đối tượng. Nó cung cấp cho ta cách tư duy trong từng tình huống của việc lập trình hướng đối tượng, và phân tích thiết kế hệ thống phần mềm. Nó cần thiết cho cả các nhà lập trình và nhà phân tích thiết kế. Đối với những người chuyên về lập trình thì việc nắm vững công cụ lập trình thôi chưa đủ, họ cần phải có một tư duy, một kỹ năng giải quyết các tình huống nhỏ của công việc xây dựng phần mềm mà họ là người thi hành. Việc giải quyết này phải đảm bảo tính ổn định là họ có thể giải quyết được trong mọi tình huống, với thời gian đúng tiến độ, phương pháp giải quyết hợp lý và đặc biệt là phải theo một chuẩn nhất định. Những nhà phân tích thiết kế mức cao, việc nắm vững công cụ lập trình có thể là không cần thiết, nhưng họ cũng cần phải biết được ở những khâu nhỏ nhất chi tiết nhất của thiết kế của họ đưa ra có thể thực hiện được hay không và nếu thực hiện được thì có thể thực hiện như thế nào, và sẽ theo một chuẩn ra sao.

Design pattern được dùng khắp ở mọi nơi, trong các phần mềm hướng đối tượng các hệ thống lớn. Trong các chương trình trò chơi, ... Và cả trong các hệ thống tính toán song song,...

Design pattern thể hiện tính kinh nghiệm của công việc lập trình, xây dựng và thiết kế phần mềm. Có thể chúng ta đã gặp design pattern ở đâu đó, trong các ứng dụng, cũng có thể chúng ta đã từng sử dụng những mẫu tương tự như design pattern để giải quyết những tình huống của mình, nhưng chúng ta không có một khái niệm gì về nó cả. Trong nội dung đề án môn học này chúng tôi xin trình bày những hiểu biết của mình về design pattern theo hướng tiếp cận mang tính kinh nghiệm. Việc cài đặt các mẫu được trình bày trên một tài liệu đi kèm.

Chúng em xin cảm ơn sự hướng dẫn của thầy Nguyễn Ngọc Bình, đã giúp đỡ chúng em hoàn thành đề án môn học này.

A. Tổng quan về Design pattern.

I. Vấn đề trong thiết kế phần mềm hướng đối tượng

Người ta nói rằng, việc thiết kế một phần mềm hướng đối tượng là một công việc khó, và việc thiết kế một phần mềm hướng đối tượng phục vụ cho mục đích dùng lại còn khó hơn. Chúng ta phải tìm ra những đối tượng phù hợp, đại diện cho một lớp các đối tượng. Sau đó thiết kế giao diện và cây kế thừa cho chúng, thiết lập mối quan hệ giữa chúng. Thiết kế của chúng ta phải đảm bảo là giải quyết được các vấn đề hiện tại, có thể tiến hành mở rộng trong tương lai mà tránh phải thiết kế lại phần mềm. Và một tiêu chí quan trọng là phải nhỏ gọn. Việc thiết kế một phần mềm hướng đối tượng phục vụ cho mục đích dùng lại là một công việc khó, phức tạp vì vậy chúng ta không thể mong chờ thiết kế của mình sẽ là đúng, và đảm bảo các tiêu chí trên ngay được. Thực tế là nó cần phải được thử nghiệm sau vài lần và sau đó nó sẽ được sửa chữa lại. Đứng trước một vấn đề, một người phân tích thiết kế tốt có thể đưa ra nhiều phương án giải quyết, anh ta phải duyệt qua tất cả các phương án và rồi chọn ra cho mình một phương án tốt nhất. Phương án tốt nhất này sẽ được anh ta dùng đi dùng lại nhiều lần, và dùng mỗi khi gặp vấn đề tương tự. Mà trong phân tích thiết kế phần mềm hướng đối tượng ta luôn gặp lại những vấn đề tương tự như nhau.

II. Lịch sử design pattern

Ý tưởng dùng mẫu xuất phát từ ngành kiến trúc, Alexander, Ishikawa, Silverstein, Jacobson, Fiksdahl-King và Angel (1977) lần đầu tiên đưa ra ý tưởng dùng các mẫu chuẩn trong thiết kế xây dựng và truyền thông. Họ đã xác định và lập sơ liệu các mẫu có liên quan để có thể dùng để giải quyết các vấn đề thường xảy ra trong thiết kế các cao ốc. Mỗi mẫu này là một cách thiết kế, chúng đã được phát triển hàng trăm năm như là các giải pháp cho các vấn đề mà người ta làm trong lĩnh vực xây dựng thường gặp. Các giải pháp tốt nhất có được ngày hôm nay là qua một quá trình sàng lọc tự nhiên. Mặc dù ngành công nghệ phần mềm không có lịch sử phát triển lâu dài như ngành kiến trúc, xây dựng nhưng Công nghệ phần mềm là một ngành công nghiệp tiên tiến, tiếp thu tất cả những gì tốt đẹp nhất từ các ngành khác. Mẫu được xem là giải pháp tốt để giải quyết vấn đề xây dựng hệ thống phần mềm.

Suốt những năm đầu 1990, thiết kế mẫu được thảo luận ở các hội thảo workshop, sau đó người ta nỗ lực để đưa ra danh sách các mẫu và lập sơ liệu về chúng. Những người tham gia bị dồn vào việc cần thiết phải cung cấp một số kiểu cấu trúc ở một mức quan niệm cao hơn đối tượng và lớp để cấu trúc này có thể được dùng để tổ chức các lớp. Đây là kết quả của sự nhận thức được rằng việc dùng các kỹ thuật hướng đối tượng độc lập sẽ không mang lại những cải tiến đáng kể đối với chất lượng cũng như hiệu quả của công việc phát triển phần mềm. Mẫu được xem là cách tổ chức việc phát triển hướng đối tượng, cách đóng gói các kinh nghiệm của những người đi trước và rất hiệu quả trong thực hành.

Năm 1994 tại hội nghị PLoP(Pattern Language of Programming Design) đã được tổ chức. Cũng trong năm này quyển sách Design patterns : Elements of Reusable Object Oriented Software (Gamma, Johnson, Helm và Vissdes, 1995) đã được xuất bản đúng vào thời điểm diễn ra hội nghị OOPSLA'94. Đây là một tài liệu còn phôi thai trong việc làm nổi bật ảnh hưởng của mẫu đối với việc phát triển phần mềm, sự đóng

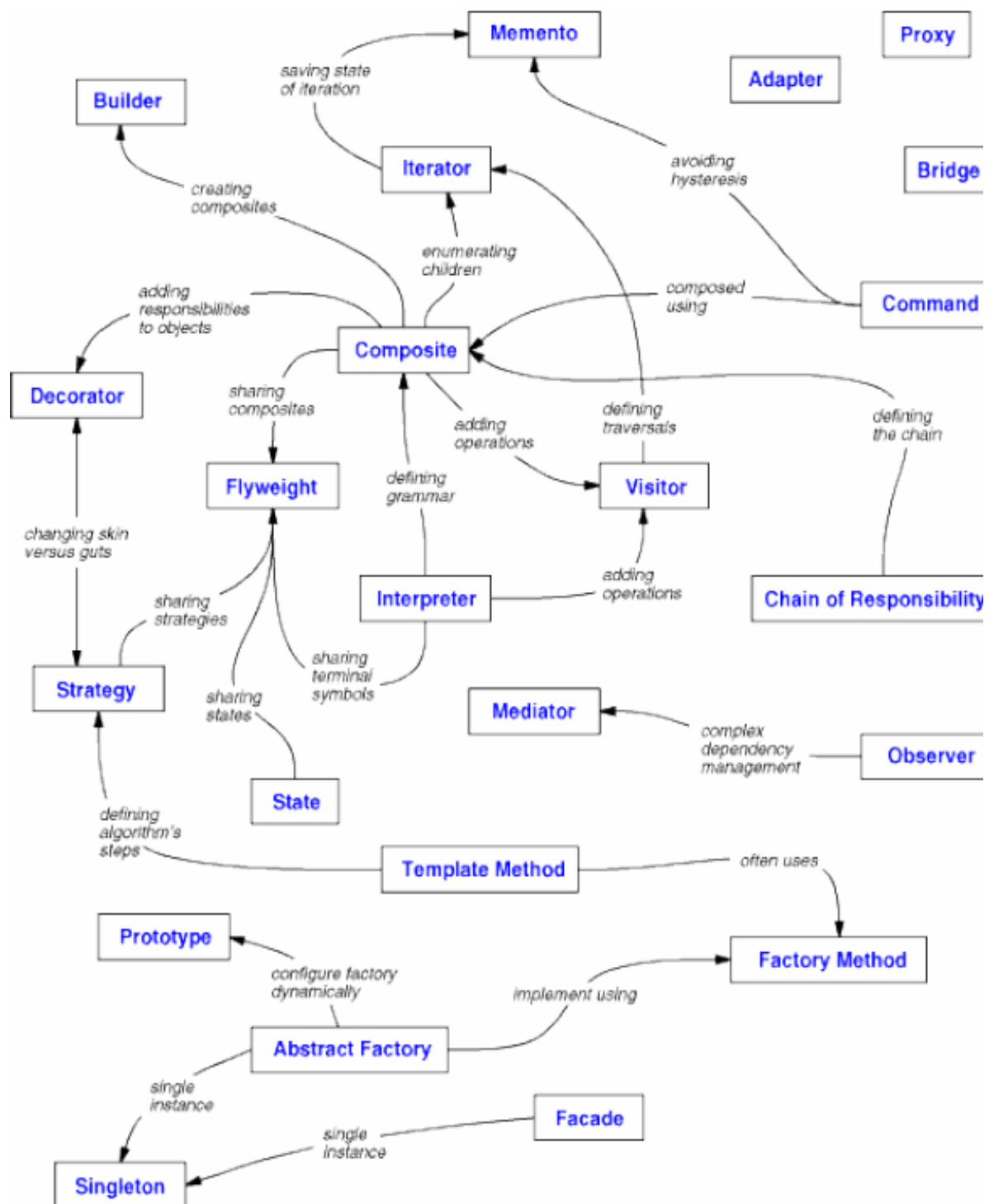
góp của nó là xây dựng các mẫu thành các danh mục (catalogue) với định dạng chuẩn được dùng làm tài liệu cho mỗi mẫu và nổi tiếng với tên Gang of Four (bộ tứ), và các mẫu nó thường được gọi là các mẫu Gang of Four. Còn rất nhiều các cuốn sách khác xuất hiện trong 2 năm sau, và các định dạng chuẩn khác được đưa ra.

Năm 2000 Evitts có tổng kết về cách các mẫu xâm nhập vào thế giới phần mềm (sách của ông lúc bấy giờ chỉ nói về những mẫu có thể được sử dụng trong UML chứ chưa đưa ra khái niệm những mẫu thiết kế một cách tổng quát). Ông công nhận Kent Beck và Ward Cunningham là những người phát triển những mẫu đầu tiên với SmallTalk trong công việc của họ được báo cáo tại hội nghị OOPSLA'87. Có 5 mẫu mà Kent Beck và Ward Cunningham đã tìm ra trong việc kết hợp các người dùng của một hệ thống mà họ đang thiết kế. Năm mẫu này đều được áp dụng để thiết kế giao diện người dùng trong môi trường Windows.

III.Design pattern là gì ?

Design patterns là tập các giải pháp cho các vấn đề phổ biến trong thiết kế các hệ thống máy tính. Đây là tập các giải pháp đã được công nhận là tài liệu có giá trị, những người phát triển có thể áp dụng giải pháp này để giải quyết các vấn đề tương tự. Giống như với các yêu cầu của thiết kế và phân tích hướng đối tượng (nhằm đạt được khả năng sử dụng các thành phần và thư viện lớp), việc sử dụng các mẫu cũng cần phải đạt được khả năng tái sử dụng các giải pháp chuẩn đối với vấn đề thường xuyên xảy ra.

Christopher Alexander nói rằng :” Mỗi một mẫu mô tả một vấn đề xảy ra lặp đi lặp lại trong môi trường và mô tả cái cốt lõi của giải pháp để cho vấn đề đó. Bằng cách nào đó bạn đã dùng nó cả triệu lần mà không làm giống nhau 2 lần”.



Mối quan hệ giữa các Pattern

Design pattern không phải là một phần của UML cốt lõi, nhưng nó lại được sử dụng rộng rãi trong thiết kế hệ thống hướng đối tượng và UML cung cấp các cơ chế biểu diễn mẫu dưới dạng đồ họa.

B. Hệ thống các mẫu design pattern.

I. Hệ thống các mẫu

Hệ thống các mẫu design pattern hiện có 23 mẫu được định nghĩa trong cuốn “Design patterns Elements of Reusable Object Oriented Software”. Hệ thống các mẫu này có thể nói là đủ và tối ưu cho việc giải quyết hết các vấn đề của bài toán phân tích thiết kế và xây dựng phần mềm trong thời điểm hiện tại. Hệ thống các mẫu design pattern được chia thành 3 nhóm: Creational, nhóm Structural, nhóm behavioral.

1. Nhóm Creational

Gồm có 5 pattern: AbstractFactory, Abstract Method, Builder, Prototype, và Singleton. Nhóm này liên quan tới việc tạo ra các thể nghiệm (instance) của đối tượng, tách biệt với cách được thực hiện từ ứng dụng. Muốn xem xét thông tin của các mẫu trong nhóm này thì phải dựa vào biểu đồ nào phụ thuộc vào chính mẫu đó, mẫu thiên về hành vi hay cấu trúc.

2. Nhóm Structural

Gồm có 7 mẫu : Adapter, Bridge, Composite, Decorator, Facade, Proxy, và Flyweight. Nhóm này liên quan tới các quan hệ cấu trúc giữa các thể nghiệm, dùng kế thừa, kết tập, tương tác. Để xem thông tin về mẫu này phải dựa vào biểu đồ lớp của mẫu.

3. Nhóm Behavioral gồm có 11 mẫu : Interpreter, Template Method, Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy và Visitor. Nhóm này liên quan đến các quan hệ gán trách nhiệm để cung cấp các chức năng giữa các đối tượng trong hệ thống. Đối với các mẫu thuộc nhóm này ta có thể dựa vào biểu đồ cộng tác và biểu đồ diễn tiến. Biểu đồ cộng tác và biểu đồ diễn tiến sẽ giải thích cho ta cách chuyển giao của các chức năng.

4. Sơ lược chuẩn của mẫu

Mẫu được phân loại thành 2 nhóm Pattern catalogue (danh mục mẫu) và pattern language (ngôn ngữ mẫu). Một pattern catalogue là một nhóm mẫu có quan hệ với nhau có thể được sử dụng cùng nhau hoặc độc lập. Một pattern language sẽ lập sơ lược mẫu cho các mẫu làm cùng nhau và có thể được áp dụng để giải quyết các vấn đề trong một lĩnh vực nào đó. Các mẫu được lập sơ lược bằng cách dùng các template, các template cung cấp các heading bên dưới có chứa chi tiết của mẫu và cách thức nó làm việc cho phép người dùng biết mẫu đó có thích hợp với vấn đề của họ hay không, nếu có thì áp dụng mẫu này để giải quyết vấn đề. Có 4 loại template khác nhau, hai trong số đó thường được sử dụng nhất là Coplien và Gamma. Các heading được liệt kê dưới đây là template của Coplien

- Name – Tên của mẫu, mô tả ý tưởng, giải pháp theo một số cách
- Problem - Vấn đề mà mẫu giúp giải quyết
- Context - Ngữ cảnh ứng dụng của mẫu (kiến trúc hoặc nghiệp vụ) và các yếu tố chính để mẫu làm việc thành công trong một tình huống nào đó.
- Force – Các ràng buộc hoặc các vấn đề phải được giải quyết bởi mẫu; chúng tạo ra sự mất cân đối, mẫu sẽ giúp ta cân đối.
- Solution - Giải pháp để cân đối các ràng buộc xung đột và làm cho hợp với ngữ cảnh
- Sketch - Bản phác thảo tượng trưng của các ràng buộc và cách giải quyết chúng.
- Resulting context - Ngữ cảnh sau khi thay đổi giải pháp.

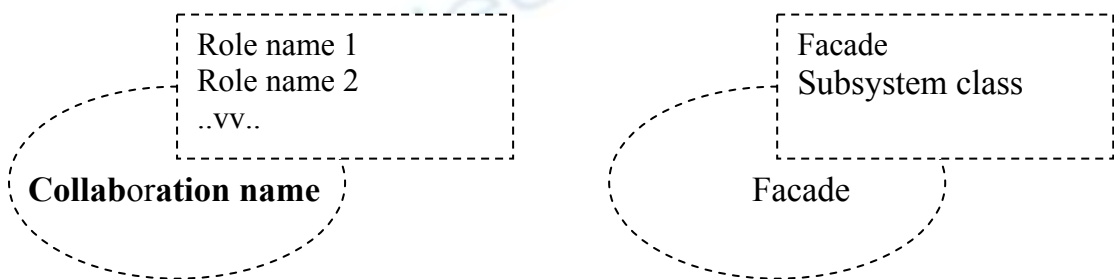
- Rationale – Lý do và động cơ cho mẫu

Sưu liệu có thể gồm mã và các biểu đồ tiêu biểu. Các biểu đồ UML có thể được dùng để minh họa cho cách làm việc của từng mẫu. Việc lựa chọn kiểu biểu đồ phụ thuộc vào bản chất của mẫu.

5. Quy tắc biểu diễn mẫu trong UML

Một trong những mục tiêu của UML là hỗ trợ các khái niệm ở cấp cao, như thành phần, cộng tác, framework và mẫu. Việc hỗ trợ này được thực hiện bằng cách cung cấp một cơ chế nhằm định nghĩa rõ ràng ngữ nghĩa của chúng, từ đó việc sử dụng các khái niệm được dễ dàng hơn nhằm đạt được khả năng tái sử dụng mà các phương pháp hướng đối tượng yêu cầu. Khía cạnh thuộc cấu trúc của mẫu được biểu diễn trong UML bằng cách dùng các cộng tác mẫu.

Cộng tác mẫu được biểu diễn bằng một hình Ellipse nét đứt và một hình chữ nhật nét đứt nằm chồng lên phần cung phía trên bên phải của nó như sau



Ký hiệu cho mẫu cộng tác

Các vai trò liên quan trong mẫu Facade

II. Nội dung các mẫu Design pattern

Nhóm Creational

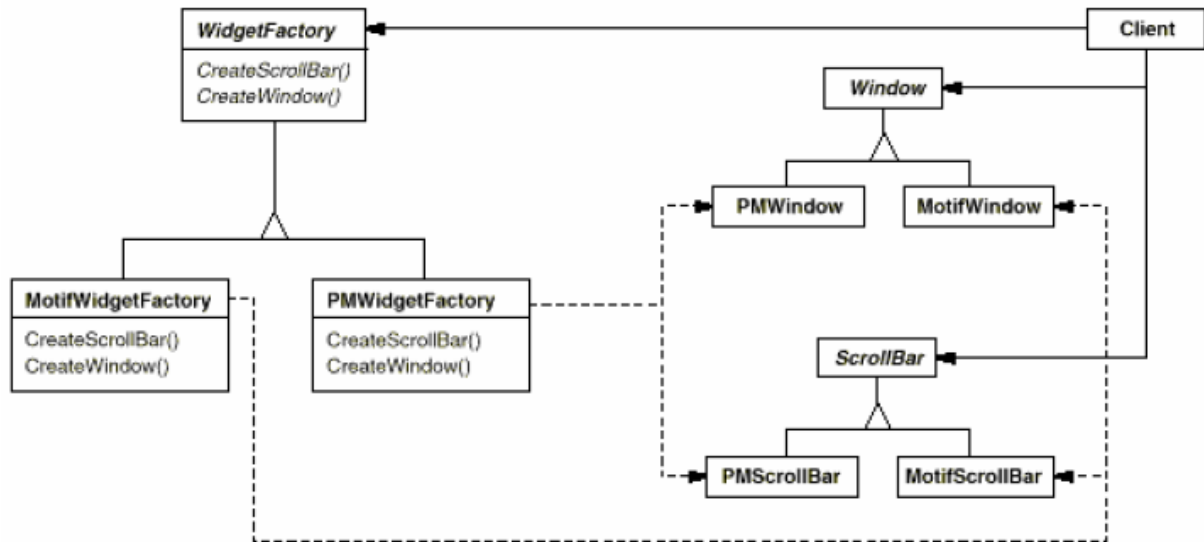
1. Abstract factory:

(tần suất sử dụng : cao trung bình)

a. Vấn đề đặt ra

Chúng ta có thể dễ dàng thấy trong các hệ điều hành giao diện đồ họa, một bộ công cụ muốn cung cấp một giao diện người dùng dựa trên chuẩn look - and - feel, chẳng hạn như chương trình trình diễn tài liệu power point. Có rất nhiều kiểu giao diện look - and - feel và cả những hành vi giao diện người dùng khác nhau được thể hiện ở đây như thanh cuộn tài liệu (scroll bar), cửa sổ (window), nút bấm (button), hộp soạn thảo (editbox),... Nếu xem chúng là các đối tượng thì chúng ta thấy chúng có một số đặc điểm và hành vi khá giống nhau về mặt hình thức nhưng lại khác nhau về cách thực hiện. Chẳng hạn đối tượng button và window, editbox có cùng các thuộc tính là chiều rộng, chiều cao, tọa độ,... Có các phương thức là Resize(), SetPosition(),... Tuy nhiên các đối tượng này không thể gộp chung vào một lớp được vì theo nguyên lý xây dựng lớp thì các đối tượng thuộc lớp phải có các phương thức hoạt động giống nhau. Trong khi ở đây tuy rằng các đối tượng có cùng giao diện nhưng cách thực hiện các hành vi tương ứng lại hoàn toàn khác nhau.

Vấn đề đặt ra là phải xây dựng một lớp tổng quát, có thể chứa hết được những điểm chung của các đối tượng này để từ đó có thể dễ dàng sử dụng lại, ta gọi lớp này là WidgetFactory. Các lớp của các đối tượng window, button, editbox kế thừa từ lớp này. Trong thiết kế hướng đối tượng, xây dựng một mô hình các lớp như thế được tối ưu hoá như sau:



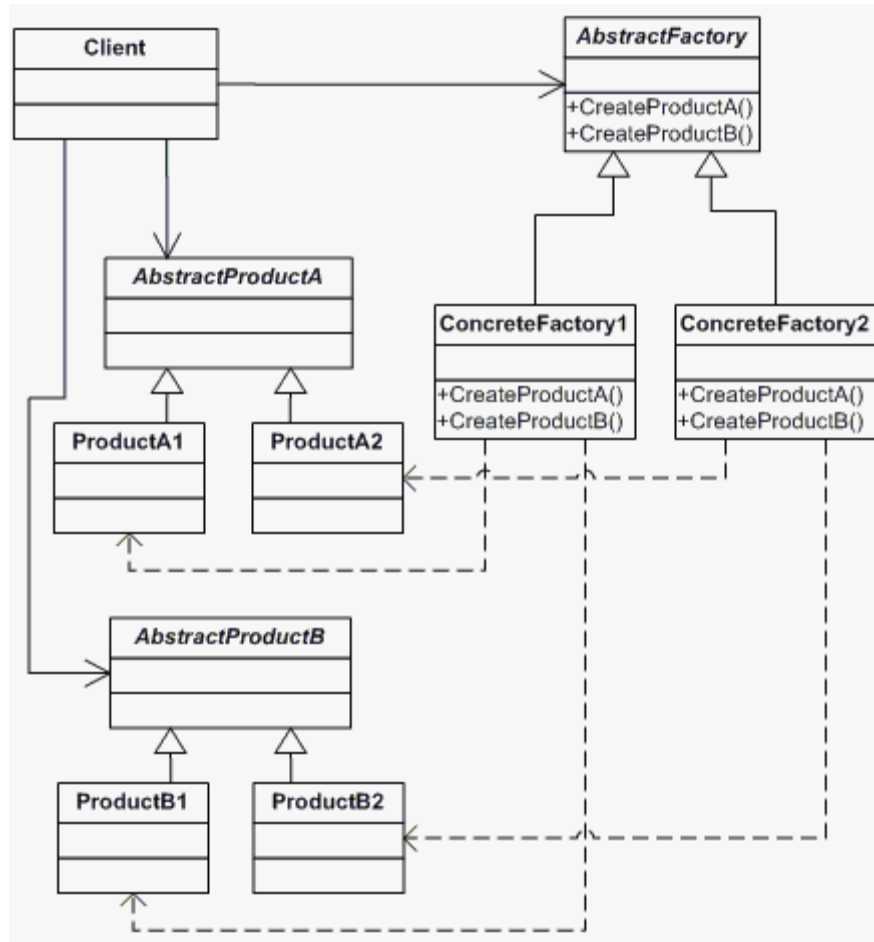
Lớp WidgetFactory có 2 phương thức là CreateScrollBar() và CreateWindow() đây là lớp giao diện trừu tượng tổng quát cho tất cả các MotifWidgetFactory và PMWidgetFactory. Các lớp

MotifWidgetFactory và PMWidgetFactory kế thừa trực tiếp từ lớp WidgetFactory. Trong sơ đồ trên còn có 2 nhóm lớp Window và ScrollBar, chúng đều là các lớp trừu tượng. Từ lớp Window sinh ra các lớp con cụ thể là PMWindow và MotifWindow. Từ lớp ScrollBar sinh ra các lớp con cụ thể là PMScrollBar và MotifScrollBar. Các đối tượng thuộc lớp này được các đối tượng thuộc lớp Factory (MotifWidgetFactory và PMWidgetFactory) gọi trong các hàm tạo đối tượng. Đối tượng trong ứng dụng (đối tượng khách - client) chỉ thông qua lớp giao diện của các đối tượng MotifWidgetFactory và PMWidgetFactory và các đối tượng trừu tượng Window và ScrollBar để làm việc với các đối tượng PMWindow, MotifWindow, PMScrollBar, MotifScrollBar. Điều này có được nhờ cơ chế binding trong các ngôn ngữ hỗ trợ lập trình hướng đối tượng như C++, C#, Java, Small Talk, ... Các đối tượng PMWindow, MotifWindow, PMScrollBar, MotifScrollBar được sinh ra vào thời gian chạy chương trình, nên trình ứng dụng (đối tượng thuộc lớp client) chỉ cần giữ một con trỏ trỏ đến đối tượng thuộc lớp WidgetFactory, và thay đổi địa chỉ trỏ đến nó có thể làm việc với tất cả các đối tượng ở trên. Những tình huống thiết kế như thế này thường có cùng một cách giải quyết đã được chứng tỏ là tối ưu. Nó được tổng quát hoá thành một mẫu thiết kế gọi là **AbstractFactory**.

b. Định nghĩa:

Mẫu AbstractFactory là một mẫu thiết kế mà cung cấp cho trình khách một giao diện cho một họ hoặc một tập các đối tượng thuộc các lớp khác nhau nhưng có cùng chung giao diện với nhau mà không phải trực tiếp làm việc với từng lớp con cụ thể.

c. Lược đồ UML



AbstractFactory (ContinentFactory)

Khai báo một giao diện cho các thao tác để tạo ra các dẫn xuất trừu tượng

ConcreteFactory (AfricaFactory, AmericaFactory)

Cài đặt các thao tác để tạo ra các đối tượng dẫn xuất chi tiết

AbstractProduct (Herbivore, Carnivore)

Khai báo một giao diện cho một kiểu đối tượng dẫn xuất

Product (Wildebeest, Lion, Bison, Wolf)

Định nghĩa một đối tượng dẫn xuất được tạo ra bởi một factory cụ thể tương ứng.

Cài đặt giao diện AbstractProduct

Client (AnimalWorld)

Sử dụng giao diện được khai báo bởi các lớp AbstractFactory và AbstractProduct

Cài đặt cho lớp WidgetFactory:

```

class WidgetFactory
{
public:
    virtual Window* CreateWindow();
    virtual ScrollBar* CreateScrollBar();
};
  
```

Cài đặt cho lớp MotifWidgetFactory và PMWidgeFactory như sau:

```
class MotifWidgetFactory:public WidgetFactory
{
public:
    Window* CreateWindow()
    {
        return new MotifWindow();
    }
    ScrollBar* CreateScrollBar()
    {
        return new MotifScrollBar();
    }
};
class PMWidgetFactory:public WidgetFactory
{
public:
    Window* CreateWindow()
    {
        return new PMWindow();
    }
    ScrollBar* CreateScrollBar()
    {
        return new PMScrollBar();
    }
};
```

Trong đó các lớp đối tượng Window được định nghĩa như sau:

```
class Window
{
    //Các thuộc tính và các phương thức tĩnh và ảo định nghĩa tại đây
};
class MotifWindow:public Window
{
    //Các thuộc tính và các phương thức định nghĩa tại đây
};
class PMWindow:public Window
{
    //Các thuộc tính và các phương thức định nghĩa tại đây
};
```

Các lớp thuộc nhóm ScrollBar.

```
class ScrollBar
{
    //Các thuộc tính và các phương thức tĩnh và ảo định nghĩa tại đây
};
class MotifScrollBar:public ScrollBar
```

```
{
    //Các thuộc tính và các phương thức định nghĩa tại đây
};
class PMScrollBar:public ScrollBar
```

```
{
    //Các thuộc tính và các phương thức định nghĩa tại đây
};
```

Để truy cập đến các đối tượng này tại chương trình ứng dụng ta có thể thực hiện như sau:

```
class Client
{
private:
    WidgetFactory* wf;
};
```

d. Mẫu liên quan:

AbstractFactory thường được cài đặt cùng với singleton, FactoryMethod đôi khi còn dùng cả Prototype. Các lớp con cụ thể (concrete class) thường được cài đặt bằng singleton. Bởi singleton có thể tạo ra những đối tượng đồng nhất cho dù chúng ta gọi nó ở đâu trong chương trình. Các mẫu này sẽ được nói kỹ hơn ở các phần sau.

2. Builder

(tần suất sử dụng : trung bình)

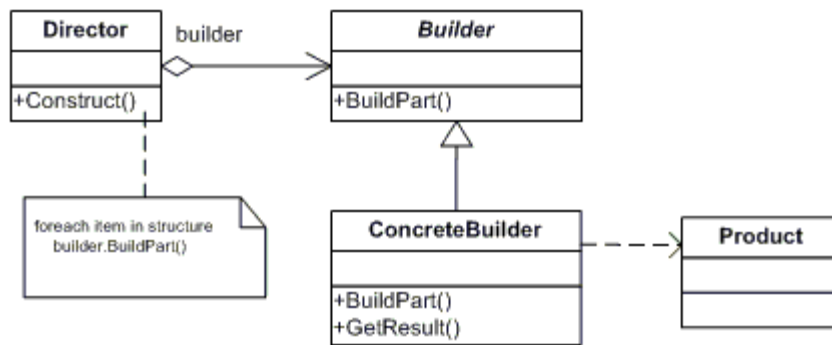
a. Vấn đề đặt ra

Trong những ứng dụng lớn, với nhiều các chức năng phức tạp và mô hình giao diện đồ sộ. Việc khởi tạo ứng dụng thường gặp nhiều khó khăn. Chúng ta không thể dồn tất cả công việc khởi tạo này cho một hàm khởi tạo. Vì như thế sẽ rất khó kiểm soát hết, và hơn nữa không phải lúc nào các thành phần của ứng dụng cũng được khởi tạo một cách đồng bộ. Có thành phần được tạo ra vào lúc dịch chương trình nhưng cũng có thành phần tùy theo từng yêu cầu của người dùng, tùy vào hoàn cảnh của ứng dụng, mà nó sẽ được tạo ra. Do vậy người ta giao công việc này cho một đối tượng chịu trách nhiệm khởi tạo, và chia việc khởi tạo ứng dụng riêng rẽ, để có thể tiến hành khởi tạo riêng biệt ở các hoàn cảnh khác nhau. Hãy tưởng tượng việc tạo ra đối tượng của ta giống như việc chúng ta tạo ra chiếc xe đạp. Đầu tiên ta tạo ra khung xe, sau đó tạo ra bánh xe, chúng ta tạo ra bu lông xe, xích, líp,.. Việc tạo ra các bộ phận này không nhất thiết phải được thực hiện một cách đồng thời hay theo một trật tự nào cả, và nó có thể được tạo ra một cách độc lập bởi nhiều người. Nhưng trong một mô hình sản xuất như vậy, bao giờ việc tạo ra chiếc xe cũng được khép kín để tạo ra chiếc xe hoàn chỉnh, đó là nhà máy sản xuất xe đạp. Ta gọi đối tượng nhà máy sản xuất xe đạp này là builder (người xây dựng).

b. Định nghĩa

Builder là mẫu thiết kế hướng đối tượng được tạo ra để chia một công việc khởi tạo phức tạp của một đối tượng ra riêng rẽ từ đó có thể tiến hành khởi tạo đối tượng ở các hoàn cảnh khác nhau.

c. Sơ đồ UML:



Builder (VehicleBuilder)

- Chỉ ra một giao diện trừu tượng cho việc tạo ra các phần của một đối tượng Product.

ConcreteBuilder (MotorCycleBuilder, CarBuilder, ScooterBuilder)

- Xây dựng và lắp ráp các phần của dẫn xuất bằng việc cài đặt bổ sung giao diện Builder.
- Định nghĩa và giữ liên kết đến đại diện mà nó tạo ra.
- Cung cấp một giao diện cho việc gọi dẫn xuất ra.

Director (Shop)

- Xây dựng một đối tượng sử dụng giao diện Builder.

Product (Vehicle)

- Biểu diễn các đối tượng phức tạp. ConcreteBuilder dựng nên các đại diện bên trong của dẫn xuất và định nghĩa quá trình xử lý bằng các thành phần lắp ráp của nó.
- Gộp các lớp mà định nghĩa các bộ phận cấu thành, bao gồm các giao diện cho việc lắp ráp các bộ phận trong kết quả cuối cùng.

d. Các mẫu liên quan

Builder thường được cài đặt cùng với các mẫu như Abstract Factory. Tầm quan trọng của Abstract Factory là tạo ra một dòng các đối tượng dẫn xuất (cả đơn giản và phức tạp). Ngoài ra Builder còn thường được cài đặt kèm với Composite pattern. Composite pattern thường là những gì mà Builder tạo ra.

3. Factory Method

(tần suất sử dụng : cao trung bình)

a. Vấn đề đặt ra

Các Framework thường sử dụng các lớp trừu tượng để định nghĩa và duy trì mối quan hệ giữa các đối tượng. Một framework thường đảm nhiệm việc tạo ra các đối tượng hoàn chỉnh. Việc xây dựng một framework cho ứng dụng mà có thể đại diện cho nhiều đối tượng tài liệu cho người dùng. Có 2 loại lớp trừu tượng chủ chốt trong framework này là lớp ứng dụng và tài liệu. Cả 2 lớp đều là lớp trừu tượng, và trình khách phải xây dựng các dẫn xuất, các lớp con để hiện thực hoá, tạo ra đối tượng phù hợp với yêu cầu của ứng dụng. Chẳng hạn để tạo ra một ứng dụng drawing, chúng ta định nghĩa một lớp DrawingApplication và một lớp DrawingDocument. Lớp ứng dụng

chịu trách nhiệm quản lý tài liệu và chúng ta sẽ tạo ra chúng khi có nhu cầu (chẳng hạn khi người dùng chọn Open hoặc New từ menu).

Bởi vì một lớp Document cá biệt như thế này được tạo ra từ các dẫn xuất của lớp Abstract Document (trong framework) để đưa ra các thể nghiệm cho một ứng dụng Drawing, lớp ứng dụng không thể biết trước được lớp dẫn xuất của Abstract Document nào sẽ được tạo ra để trình bày, mà nó chỉ biết khi nào một đối tượng tài liệu nào nên được tạo chứ không biết loại đối tượng tài liệu nào để tạo. Điều này tạo ra một sự tiến thoái lưỡng nan: framework phải thể nghiệm một lớp, nhưng nó chỉ biết về lớp trừu tượng của nó, mà lớp trừu tượng này lại không thể thể nghiệm trực tiếp được. Nếu ai từng làm việc với giao diện ứng dụng đơn tài liệu và đa tài liệu trong ngôn ngữ Visual C++, chúng ta cũng sẽ bắt gặp một vấn đề tương tự. Đối tượng MainFrame có thể tạo ra một đối tượng view mỗi khi người dùng nhấn chuột vào menu View hay Open, nhưng MainFrame hoàn toàn không hề biết một chút thông tin nào trước đó về View vì nó không chứa bất cứ một thể nghiệm nào của View.

Mẫu Abstract Method đưa ra một giải pháp cho vấn đề này. Nó đóng gói thông tin về lớp dẫn xuất Document nào được tạo và đưa ra ngoài framework.

Lớp dẫn xuất ứng dụng định nghĩa lại một phương thức trừu tượng CreateDocument() trên lớp Application để trả về một đối tượng thuộc lớp dẫn xuất của lớp document.

Class Application

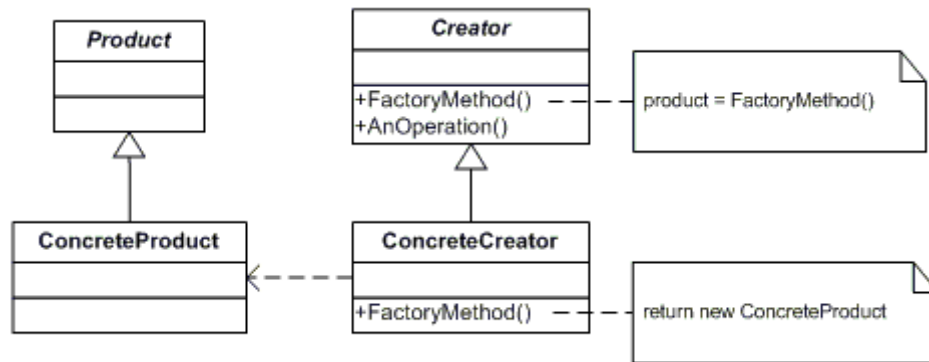
```
{
    //Các khai báo khác
Public:
    Document* CreateDocument();
    //Các khai báo khác
};
Document* Application::CreateDocument()
{
    Document* newDocument = new MyDocument();
    Return newDocument;
}
```

Khi một đối tượng thuộc lớp dẫn xuất của Application được tạo ra, nó có thể tạo ra luôn các đối tượng tài liệu đặc biệt cho ứng dụng mà không cần biết về các lớp đó. Chúng ta gọi CreateDocument là một factory method bởi vì nhiệm vụ của nó là sản xuất ra các đối tượng.

b. Định nghĩa Factory Method

Factory Method là một giao diện cho việc tạo ra một đối tượng, nhưng để cho lớp dẫn xuất quyết định lớp nào sẽ được tạo. Factory method để cho một lớp trì hoãn sự thể nghiệm một lớp con.

c. Sơ đồ UML



Product (Page)

- Định nghĩa giao diện của các đối tượng mà Factory Method tạo ra.

ConcreteProduct (SkillsPage, EducationPage, ExperiencePage)

- Cài đặt giao diện Product.

Creator (Document)

- Khai báo Factory Method mà trả về một đối tượng của kiểu Product. Sự kiến tạo này cũng có thể định nghĩa một cài đặt mặc định của Factory Method trả về một đối tượng ConcreteProduct mặc định.
- Có thể gọi Factory Method để tạo ra một đối tượng Product.

ConcreteCreator (Report, Resume)

- Chồng lên Factory Method để trả về một thể nghiệm của một ConcreteProduct.

d. Mẫu liên quan

Abstract Factory thường được cài đặt cùng với Factory Method. Lớp Factory Method thường được gọi là Template Method. Trong ví dụ về ứng dụng Drawing trên, NewDocument là một template method.

Prototype không cần đến một lớp con Creator, tuy nhiên thường đòi hỏi một phương thức để tạo thể nghiệm trên lớp dẫn xuất.

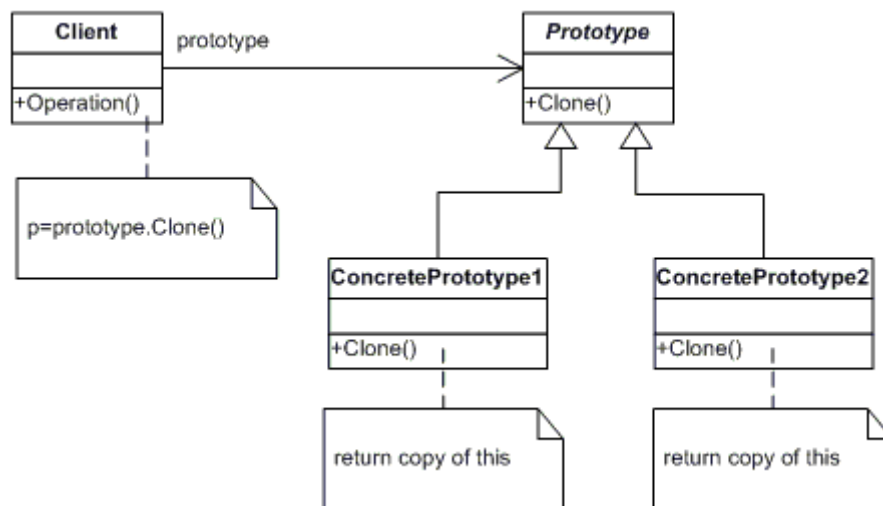
4. Prototype

(tần suất sử dụng : thấp trung bình)

a. Định nghĩa

Prototype là mẫu thiết kế chỉ định ra một đối tượng đặc biệt để khởi tạo, nó sử dụng một thể nghiệm sơ khai rồi sau đó sao chép ra các đối tượng khác từ mẫu đối tượng này.

b. Sơ đồ UML



Prototype (ColorPrototype)

- Khai báo một giao diện cho dòng vô tính của chính nó.

ConcretePrototype (Color)

- Cài đặt một thao tác cho dòng vô tính của chính nó.

Client (ColorManager)

- Tạo ra một đối tượng mới bằng việc yêu cầu một nguyên mẫu từ dòng vô tính của nó

c. Mẫu liên quan

Prototype và Abstract Factory liên quan đến nhau chặt chẽ, có thể đối chọi nhau theo nhiều kiểu. Tuy nhiên chúng cũng có thể kết hợp cùng nhau. Một Abstract Factory có thể chứa một tập các Prototype vô tính và trả về các đối tượng sản xuất.

5. Singleton

(tần suất sử dụng :Cao)

a. Vấn đề đặt ra

Ta hãy xem xét về một đối tượng quản lý tài nguyên trong các ứng dụng. Mỗi ứng dụng có một bộ quản lý tài nguyên, nó cung cấp các điểm truy cập cho các đối tượng khác trong ứng dụng. Các đối tượng (ta gọi là đối tượng khách) có thể thực hiện lấy ra từ bộ quản lý tài nguyên những gì chúng cần và thay đổi giá trị nằm bên trong bộ quản lý tài nguyên đó. Để truy cập vào bộ quản lý tài nguyên đối tượng khách cần phải có một thể nghiệm của bộ quản lý tài nguyên, như vậy trong một ứng dụng sẽ có rất nhiều thể nghiệm của bộ quản lý tài nguyên được tạo ra.

class ResourceManager

```

{
private:
    int x;
public:
    ResourceManager(){x =0;....}
    void SetX(int _x){ x = _x;}
    void GetX(){ return x;}
}
  
```

class ClientObject1

```

{
public:
    void DoSomething()
    {
        ResourceManager rm;
        printf("x = %d",rm.GetX());
        x = 100;
    }
}
class ClientObject2
{
public:
    void DoSomething()
    {
        ResourceManager rm;
        printf("x = %d",rm.GetX());
        x = 500;
    }
}

```

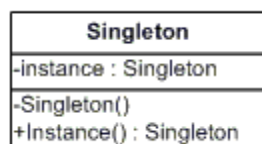
Trong ví dụ trên hàm DoSomething() của ClientObject1 khi truy cập vào đối tượng thuộc lớp ResourceManager sẽ in ra màn hình X = 0; và đặt vào đó x = 100;

Sau đó hàm Dosomething() của ClientObject2 khi truy cập vào đối tượng thuộc lớp ResourceManager sẽ in ra màn hình X = 0 và đặt vào đó x = 500; Rõ ràng là tài nguyên mà các đối tượng khách truy cập vào không thể hiện sự thống nhất lẫn nhau. Điều mà lẽ ra khi giá trị x trong ResourceManager bị ClientObject1 thay đổi thì ClientObject2 phải nhận biết được điều đó và in ra màn hình X=100 . Nhưng theo logic cài đặt ở trên thì khi đối tượng thuộc lớp ClientObject1 truy cập đến ResourceManager nó tạo ra một Instance và đặt thuộc tính x = 100; Đối tượng ClientObject2 truy cập đến ResourceManager nó tạo ra một Instance và đặt thuộc tính x = 500. Hai Instance này hoàn toàn độc lập nhau về vùng nhớ do đó mà tài nguyên x mà nó quản lý cũng là 2 tài nguyên độc lập với nhau. Vấn đề đặt ra là phải tạo ra một bộ quản lý tài nguyên hoàn hảo tạo ra mọi thể nghiệm giống nhau tại nhiều nơi nhiều thời điểm khác nhau trong ứng dụng.Singleton cung cấp cho ta một cách giải quyết vấn đề này.

b.Định nghĩa

Singleton là mẫu thiết kế nhằm đảm bảo chỉ có duy nhất một thể nghiệm và cung cấp điểm truy cập của nó một cách thống nhất toàn cục.

c.Sơ đồ UML



Singleton (LoadBalancer)

- Định nghĩa một thao tác tạo thể nghiệm cho phép đối tượng khách truy cập đến thể nghiệm đồng nhất của nó như một thao tác của lớp.

- Chịu trách nhiệm cho việc tạo ra và duy trì thể nghiệm đồng nhất của chính nó.

d. Mẫu liên quan

Có rất nhiều mẫu có thể cài đặt bổ sung từ việc sử dụng Singleton, chẳng hạn như Abstract Factory, Builder, và Prototype.

Tổng kết về nhóm Creational Pattern

Có 2 cách thông dụng nhất để tham số hoá một hệ thống bằng các lớp của đối tượng mà nó tạo. Một cách là chia nhỏ nó thành các lớp con để tạo đối tượng tương ứng. Điều này tương đương với việc chúng ta sử dụng mẫu Factory Method. Điểm hạn chế chính của cách tiếp cận này là đòi hỏi phải tạo một lớp mới khi thay đổi lớp dẫn xuất. Giống như kiểu thác nước. Chẳng hạn một dẫn xuất tự tạo ra bằng Abstract Factory, sau đó chúng ta phải đề lên lớp khởi tạo này.

Nhóm Structural

6. Adapter.

(tần suất sử dụng :cao trung bình)

a. Vấn đề đặt ra

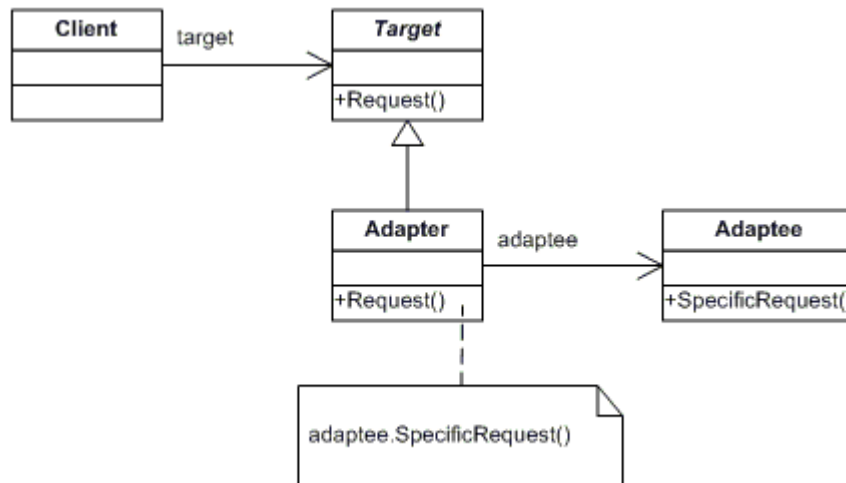
Đôi khi một lớp công cụ được thiết kế cho việc sử dụng lại, lại không thể sử dụng lại chỉ bởi giao diện không thích hợp với miền giao diện đặc biệt mà một ứng dụng yêu cầu. Adapter đưa ra một giải pháp cho vấn đề này.

Trong một trường hợp khác ta muốn sử dụng một lớp đã tồn tại và giao diện của nó không phù hợp với giao diện của một lớp mà ta yêu cầu. Ta muốn tạo ra một lớp có khả năng được dùng lại, lớp đó cho phép kết hợp với các lớp không liên quan hoặc không được dự đoán trước, các lớp đó không nhất thiết phải có giao diện tương thích với nhau. (Chỉ với đối tượng adapter) ta cần sử dụng một vài lớp con đã tồn tại, nhưng để làm cho các giao diện của chúng tương thích với nhau bằng việc phân lớp các giao diện đó là việc làm không thực tế, để làm được điều này ta dùng một đối tượng adapter để biến đổi giao diện lớp cha của nó.

b. Định nghĩa

Adapter là mẫu thiết kế dùng để biến đổi giao diện của một lớp thành một giao diện khác mà clients yêu cầu. Adapter ngăn cản các lớp làm việc cùng nhau đó không thể làm bằng cách nào khác bởi giao diện không tương thích.

c. Sơ đồ UML



Các thành phần tham gia:

- Target: định nghĩa một miền giao diện đặc biệt mà Client sử dụng.
- Client: cộng tác với các đối tượng tương thích với giao diện Target.
- Adaptee: định nghĩa một giao diện đã tồn tại mà cần phải làm biến đổi cho thích hợp.
- Adapter: làm tương thích giao diện của Adaptee với giao diện của Target.

d. Mẫu liên quan

Bridge có một cấu trúc tương tự như một đối tượng của Adapter, nhưng Bridge có một mục đích khác. Nó chia giao diện từ các phần cài đặt của nó ra riêng rẽ để từ đó có thể linh hoạt hơn và độc lập nhau. Sử dụng một Adapter đồng nghĩa với việc thay đổi giao diện của một đối tượng đã tồn tại.

Decorator nâng cấp một đối tượng khác mà không làm thay đổi giao diện của nó. Một Decorator do đó mà trong suốt với ứng dụng hơn là một Adapter. Như một hệ quả Decorator hỗ trợ cơ chế kết tập đệ quy mà điều này không thể thực hiện được đối với các Adapter thuần túy.

Proxy định nghĩa một đại diện cho một đối tượng khác và không làm thay đổi giao diện của nó.

7. Bridge

(Tần suất sử dụng : trung bình)

a. Vấn đề đặt ra

Khi một lớp trừu tượng (abstraction) có thể có một vài thành phần bổ sung thêm thì cách thông thường phù hợp với chúng là sử dụng kế thừa. Một lớp trừu tượng định nghĩa một giao diện cho trừu tượng đó, và các lớp con cụ thể thực hiện nó theo các cách khác nhau. Nhưng cách tiếp cận này không đủ mềm dẻo. Sự kế thừa ràng buộc một thành phần bổ sung thêm là cố định cho abstraction, điều này làm nó khó thay đổi, mở rộng, và sử dụng lại các abstraction, các thành phần bổ sung một cách độc lập. Trong trường hợp này dùng một mẫu Bridge là thích hợp nhất. Mẫu Bridge thường được ứng dụng khi :

- Ta muốn tránh một ràng buộc cố định giữa một abstraction và một thành phần bổ sung thêm của nó.

- Cả hai, các abstraction và các thành phần cài đặt của chúng nên có khả năng mở rộng bằng việc phân chia lớp. Trong trường hợp này, Bridge pattern cho phép ta kết hợp các abstraction và các thành phần bổ sung thêm khác nhau và mở rộng chúng một cách độc lập.

- Thay đổi trong thành phần được bổ sung thêm của một abstraction mà không ảnh hưởng đối với các client, tức là mã của chúng không nên đem biên dịch lại.

- Ta muốn làm ẩn đi hoàn toàn các thành phần bổ sung thêm của một abstraction khỏi các client.

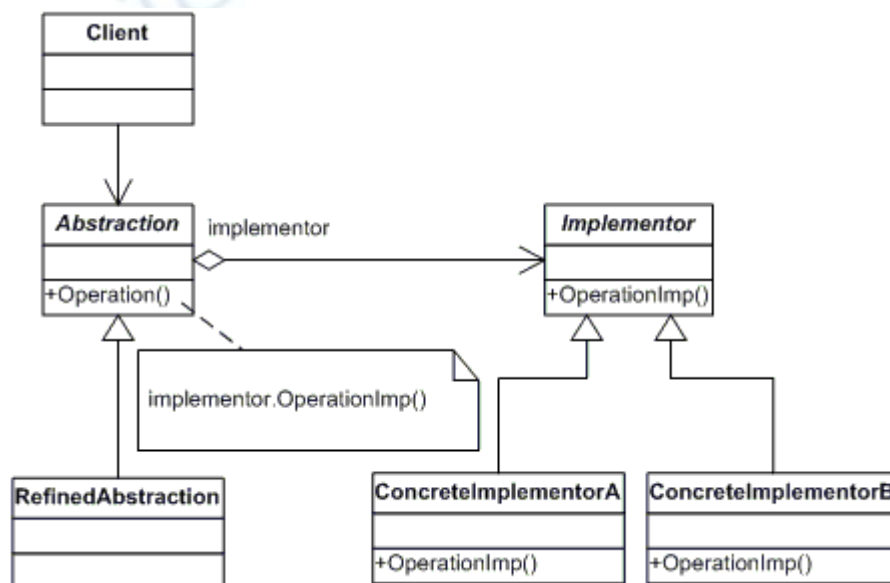
- Ta có một sự phát triển rất nhanh các lớp, hệ thống phân cấp lớp chỉ ra là cần phải tách một đối tượng thành hai phần.

- Ta muốn thành phần bổ sung thêm có mặt trong nhiều đối tượng, và việc này lại được che khỏi client (client không thấy được).

b. Định nghĩa

Bridge là mẫu thiết kế dùng để tách riêng một lớp trừu tượng khỏi thành phần cài đặt của nó để có được hai cái có thể biến đổi độc lập.

c. Sơ đồ UML



Abstraction (BusinessObject)

- Định nghĩa một giao diện trừu tượng

- Duy trì một tham chiếu tới đối tượng của các lớp kế thừa từ nó

RefinedAbstraction (CustomersBusinessObject)

- Mở rộng giao diện bằng cách định nghĩa một đối tượng trừu tượng

Implementor (DataObject)

- Định nghĩa giao diện cho lớp kế thừa. Giao diện này không phải tương ứng chính xác với giao diện trừu tượng. Trong thực tế 2 giao diện này có thể khá là độc lập. Việc kế thừa một cách tùy ý các giao diện cũng chỉ cung cấp duy nhất các thao tác nguyên thủy và lớp trừu tượng định nghĩa một thao tác mức trên dựa những thao tác nguyên thủy này.

ConcreteImplementor (CustomersDataObject)

- Cài đặt giao diện đã được cài đặt và định nghĩa một cài đặt cụ thể.